
Games Engineering

Dong, Wei
5726890

University of Warwick

September 18, 2026

<https://github.com/WeiD-Warwick/WM9M4-Rasterizer>
<https://github.com/WeiD-Warwick/WM9M4-ChatRoom>

1 Rasterizer Optimization

1.1 Overview

1.1.1 Why do I Choose My Scene 3

In the current implementation, Scene 1 and Scene 2 do not show significant improvement in view frustum culling. Therefore, I designed Scene 3 to test optimizations in view frustum culling.

In Scene 3, four mesh cubes are generated at the four edges of the view frustums. I intentionally ensured that half of these meshes were in-screen and half were out-of-screen. This generates a large amount of triangle/pixel workload, and since some objects are located at the edges of the view frustums, clipping/boundary checks are easily triggered.

Because Scene 1 and Scene 2 cannot effectively stress or expose the impact of view frustum culling strategies, Scene 3 can be used to evaluate the benefits of my renderer's optimizations such as multi-threaded parallelization, view frustum/off-screen processing.

1.1.2 Optimization Results

The performance improvements from this optimization are mainly concentrated in the following three areas: rendering pipeline-level optimization, SIMD vectorization, and multi-threaded parallelization. All three yielded substantial benefits.

Rendering pipeline-level optimization: Techniques such as vertex caching, edge function scanning, early Z-testing, backface culling, and inverse

area precomputation to reduce redundant calculations per pixel.

SIMD vectorization: AVX SIMD instructions are used to batch vertex processing and lighting calculations, improving arithmetic throughput and data-level parallelism.

Multi-threaded parallelization:

Achieving parallel rendering based on tile-based task partitioning and thread pool distribution to reduce single-thread bottlenecks. The overall goal is to improve rendering throughput per frame and maintain stable performance scalability as scene complexity increases.

1.2 Single-Thread Optimizations

1.2.1 Small Optimizations

This section focuses on optimizations that involve minor changes but offer significant performance improvements.

VertexCache: Reduces redundant vertex processing, improving geometry processing efficiency.

Light Pre-normalize: Pre-processes lighting vectors, reducing per-pixel/per-vertex overhead.

Inverse Area: Avoids redundant division operations, improving interpolation/rasterization efficiency.

Early Z-Testing: Performs depth testing in advance, reducing invalid pixel shading and improving pixel-level efficiency.

Core Code

```
1 void preProcessVertexCache(matrix& p, float w,
2   float h, std::vector<Vertex>& vcache) {
3   vcache.resize(vertices.size());
4   for (unsigned int i = 0; i < vertices.size(); ++i) {
5       Vertex out;
6       out.p = p * vertices[i].p;
7       out.p.divideW();
```

```

8         out.normal = world * vertices[i].
normal;
9         out.normal.normalise();
10
11         // Map NDC -> screen
12         out.p[0] = (out.p[0] + 1.f) * halfW;
13         out.p[1] = h - (out.p[1] + 1.f) *
halfH;
14
15         out.rgb = vertices[i].rgb;
16         vcache[i] = out;
17     }
18 }
19
20 #if OPT_EARLY_Z_TEST
21 float depth = interpolate(alpha, beta, gamma, v
[0].p[2], v[1].p[2], v[2].p[2]);
22 if (renderer.zbuffer(x, y) <= depth || depth <=
0.001f) continue;
23
24 // Interpolate color, depth, and normals
25 colour c = interpolate(alpha, beta, gamma, v[0].
rgb, v[1].rgb, v[2].rgb);
26 c.clampColour();
27 #else
28
29     triangle(const Vertex& v1, const Vertex& v2,
const Vertex& v3) {
30         v[0] = v1;
31         v[1] = v2;
32         v[2] = v3;
33
34         // Calculate the 2D area of the triangle
35         vec2D e1 = vec2D(v[1].p - v[0].p);
36         vec2D e2 = vec2D(v[2].p - v[0].p);
37
38     #if OPT_BACKFACE_CULLING
39         signedArea = (e1.x * e2.y - e1.y * e2.x)
;
40         area = std::fabs(signedArea);
41     #else
42         area = std::fabs(e1.x * e2.y - e1.y * e2
.x);
43     #endif
44
45     #if OPT_INV_AREA
46         invArea = (area > 0.f) ? (1.0f / area) :
0.f;
47     #endif
48 }
49

```

Listing 1: Message Header Struct

Result

These four optimizations (Vertex Cache, Light Pre-normalize, Inverse Area, and Early Z-Testing) all delivered positive performance improvements in all three scenes, with increased FPS and decreased P99 latency, indicating higher frame rates and better stability.

The speedup ratio ranged from 1.09 to 1.13, representing small but stable performance improved.

Scene 1

FPS from 267.20 to 299.14 (+12.0%)

P99 from 6.17 to 5.58 (-9.6%)

Scene 2

FPS from 120.32 to 134.14 (+11.5%)

P99 from 10.37 to 9.11 (-12.1%)

Scene 3

FPS from 79.22 to 87.16 (+10.0%)

Table 1: Performance between baseline and small optimizations

Version	FPS	P99	Speedup
Baseline Scene1	267.20	6.17	1
Optimized Scene1	299.14	5.58	1.1
Baseline Scene2	120.32	10.37	1
Optimized Scene2	134.14	9.11	1.13
Baseline Scene3	79.22	14.76	1
Optimized Scene3	87.16	13.49	1.09

Table 2: Performance between baseline and Backface Culling

Version	FPS	P99	Speedup
Baseline Scene1	267.20	6.17	1
BC Scene1	355.84	4.82	1.28
Baseline Scene2	120.32	10.37	1
BC Scene2	164.02	7.30	1.46
Baseline Scene3	79.22	14.76	1
BC Scene3	109	10.90	1.35

P99 from 14.76 to 13.49 (-8.6%)

1.2.2 Backface Culling

Triangles facing away from the camera are invisible. Back-to-back culling can optimize invalid triangles and improve rendering efficiency.

Core Code

```

1     #if OPT_BACKFACE_CULLING
2     if (signedArea <= 0.f) return;
3     #endif

```

Listing 2: Message Header Struct

Result

Backface culling resulted in a performance improvement of 1.28 to 1.46 times, significantly higher than the previous four smaller optimizations (1.09 to 1.13 times). This demonstrates that geometry culling optimization has a significant impact on overall performance.

Scene 1

FPS from 267.20 to 355.84 (+33.2%)

P99 from 6.17 to 4.82 (-21.9%)

Scene 2

FPS from 120.32 to 164.02 (+36.3

P99 from 10.37 to 7.30 (-29.6

Scene 3

FPS from 79.22 to 109.00 (+37.6%)

P99 from 14.76 to 10.90 (-26.1%)

1.2.3 Edge Function Rasterization

The current system uses a cumbersome calculation of the triangle centroid function, requiring frequent dot product operations. Therefore, an edge function is introduced to simplify the calculation of the triangle centroid.

Core Code

```

1 struct EdgeFunction {
2     float a, b, c;
3
4     EdgeFunction(float x1, float y1, float x2,
5         float y2) {
6         a = y1 - y2;
7         b = x2 - x1;
8         c = x1 * y2 - x2 * y1;
9     }
10    inline float evaluate(float x, float y)
11    const {
12        return a * x + b * y + c;
13    }
14    inline float stepX() const { return a; }
15    inline float stepY() const { return b; }
16 };
17
18 struct TriangleEdgeFunctions {
19     EdgeFunction ef0; // (v1->v2)
20     EdgeFunction ef1; // (v2->v0)
21     EdgeFunction ef2; // (v0->v1)
22
23     float rowE0 = 0.f;
24     float rowE1 = 0.f;
25     float rowE2 = 0.f;
26
27     TriangleEdgeFunctions(const vec4& v0, const
28         vec4& v1, const vec4& v2)
29         : ef0(v1[0], v1[1], v2[0], v2[1]),
30           ef1(v2[0], v2[1], v0[0], v0[1]),
31           ef2(v0[0], v0[1], v1[0], v1[1]) {}
32
33     inline void beginRow(int minX, int minY) {
34         const float fx = minX + 0.5f;
35         const float fy = minY + 0.5f;
36         rowE0 = ef0.evaluate(fx, fy);
37         rowE1 = ef1.evaluate(fx, fy);
38         rowE2 = ef2.evaluate(fx, fy);
39     }
40
41     inline void stepRow() {
42         rowE0 += ef0.b;
43         rowE1 += ef1.b;
44         rowE2 += ef2.b;
45     }
46
47     inline void getRowStart(float& e0, float& e1,
48         float& e2) const {
49         e0 = rowE0;
50         e1 = rowE1;
51         e2 = rowE2;
52     }
53
54     inline void stepPixel(float& e0, float& e1,
55         float& e2) const {
56         e0 += ef0.a;
57         e1 += ef1.a;
58         e2 += ef2.a;
59     }
60 };

```

Listing 3: Message Header Struct

Result

Edge function optimization significantly improved performance in all three scenarios (1.48 to 1.67). Compared to traditional centroid calculation methods, edge function has a huge performance improvement effect on the rasterization stage.

Scene 1

Table 3: Performance between baseline and Edge Function and all optimization

Version	FPS	P99	Speedup
Baseline 1	267.20	6.17	1
Edge 1	441.80	3.70	1.67
All OPT 1	597.09	2.90	2.12
Baseline 2	120.32	10.37	1
Edge 2	202.78	5.90	1.75
All OPT 2	266.84	4.92	2.11
Baseline 3	79.22	14.76	1
Edge 3	120.33	9.98	1.48
All OPT 3	156.68	8.52	1.73

FPS from 267.20 to 441.80 (+65%) P99 from 6.17 to 3.70 (-40%)

Scene 2

FPS from 120.32 to 202.78 (+68.5%)

P99 from 10.37 to 5.90 (-43%)

Scene 3

FPS from 79.22 to 120.33 (+51.9%)

P99 from 14.76 to 9.98 (-32.4%)

1.2.4 SIMD Acceleration (8-Pixel Parallel Shading)

At this point, the optimization limit of one pixel at a time has been reached. Therefore, SIMD was introduced to process multiple pixels simultaneously with a single instruction, thereby reducing the cost of pixel-by-pixel computation and improving the overall throughput.

Core Code

```

1 inline void mat4_mul_vec4(const float* m16,
2     const Vec4SOA& in, int startIndex, int range
3     , Vec4SOA& out) {
4
5     const int end = startIndex + range;
6     if (out.size() < end) out.resize(end);
7
8     // Matrix rows
9     __m256 m00 = _mm256_set1_ps(m16[0]);
10    __m256 m01 = _mm256_set1_ps(m16[1]);
11    __m256 m02 = _mm256_set1_ps(m16[2]);
12    __m256 m03 = _mm256_set1_ps(m16[3]);
13
14    // calculate color
15    auto calc = [&](const __m256& c0, const
16        __m256& c1, const __m256& c2, const __m256&
17        lCol, const __m256& amb) {
18        __m256 color = _mm256_add_ps(
19            _mm256_add_ps(_mm256_mul_ps(c0, alpha),
20                _mm256_mul_ps(c1, beta)), _mm256_mul_ps(c2,
21                    gamma));
22        return _mm256_min_ps(_mm256_add_ps(
23            _mm256_mul_ps(_mm256_mul_ps(color, lp.kd),
24                _mm256_mul_ps(lCol, dot)), amb),
25            _mm256_set1_ps(1.0f));
26    };
27
28    __m256_store_ps(rOut, calc(tc.r0, tc.r1,
29        tc.r2, lp.lR, lp.ambR));
30    __m256_store_ps(gOut, calc(tc.g0, tc.g1,
31        tc.g2, lp.lG, lp.ambG));

```

Table 4: Performance between baseline and SIMD + all optimization

Version	FPS	P99	Speedup
Baseline 1	267.20	6.17	1
SIMD + Other 1	1061.76	1.57	3.92
Baseline 2	120.32	10.37	1
SIMD + Other 2	550.48	2.54	4.08
Baseline 3	79.22	14.76	1
SIMD + Other 3	234.61	5.53	2.66

```

21     mm256_store_ps(bOut, calc(tc.b0, tc.b1,
22     tc.b2, lp.lB, lp.ambB));
    }

```

Listing 4: Message Header Struct

Result SIMD + other optimizations delivers significant performance improvements (2.66 to 4.08) This shows that SIMD significantly improves throughput for critical paths such as rasterization.

Scene 1

FPS from 267.20 to 1061.76 (+297%)

P99 from 6.17 to 1.57 (-74.6%)

Scene 2

FPS from 120.32 to 550.48 (+357%)

P99 from 10.37 to 2.54 (-75.5%)

Scene 3

FPS from 79.22 to 234.61 (+196%)

P99 from 14.76 to 5.53 (-62.5%)

1.3 Multi-Thread Optimizations

Before implementing multi-threaded calls, I divided the screen into tiles of fixed size, generating a job for each tile, and then submitted them in batches to a thread pool for execution. This way, each job is responsible for only a specific rectangular area of the screen (ScissorRect). When drawing triangles, the actual drawing area of the triangle is clipped to the current tile's scissor boundaries (minX/minY/maxX/maxY), ensuring that the job only writes to pixels within the tile.

Furthermore, within each tile job, threads share the VertexSOA cache, lighting parameters, mesh triangle indices, etc. These objects are passed to each job via shared_ptr and are read-only accessed within the job, preventing modification by other threads.

In summary, to avoid:

1. I used tile partitioning and scissors to limit the write range of each thread, avoiding race conditions caused by multiple threads writing to the same pixel.
2. For shared data processing, shared read-only access and write isolation are implemented.

File size	thread		FPS	P99	FRUSTUM_CULLING FPS	FRUSTUM_CULLING P99	RenderScene FPS	RenderScene P99
256x256	2	scene1	1272.37	1.58	1331.68	1.36	1335.28	1.58
		scene2	772.16	1.85	758.00	1.58	805.52	1.76
		scene3	158.83	7.78	260.17	4.92	308.38	4.49
4	4	scene1	1294.67	1.30	1461.05	1.08	1428.24	1.08
		scene2	928.90	1.43	907.50	1.52	923.34	1.63
		scene3	122.82	12.16	226.39	6.69	266.57	3.67
6	6	scene1	1240.88	1.20	1450.28	1.09	1440.23	1.09
		scene2	1001.60	1.25	1003.81	1.28	1009.29	1.24
		scene3	90.30	16.35	185.01	7.85	416.55	3.40
8	8	scene1	1085.53	2.23	1432.13	1.08	1456.58	1.03
		scene2	898.46	2.49	1003.67	1.41	1007.26	1.21
		scene3	58.54	28.22	135.31	10.63	436.30	3.26
10	10	scene1	985.43	1.99	1208.54	1.52	1428.17	1.10
		scene2	886.98	2.64	857.13	2.53	1112.43	1.38
		scene3	46.54	36.79	131.39	9.80	435.14	3.20
32x32x4	2	scene1	1101.81	1.30	1363.80	1.03	1376.90	1.14
		scene2	794.70	1.94	1104.07	1.21	821.58	1.76
		scene3	222.10	5.83	185.30	9.84	375.99	3.91
4	4	scene1	1464.42	1.08	1576.47	1.01	1503.69	1.01
		scene2	1028.20	1.40	1018.70	1.47	971.77	1.44
		scene3	148.70	10.81	296.41	5.57	452.44	2.87
6	6	scene1	1388.79	1.07	1588.73	1.01	1477.70	1.00
		scene2	1131.70	1.59	1112.41	1.25	958.30	1.43
		scene3	118.30	12.42	200.00	6.57	490.27	3.11
8	8	scene1	1256.31	1.38	1534.85	0.98	1536.62	0.95
		scene2	1134.27	1.14	1040.04	2.31	957.16	1.44
		scene3	73.62	16.45	173.74	9.42	450.10	3.10

Figure 1: Multi Thread Speedup**1.3.1 Frustum Culling**

Before multi-threaded rendering, objects outside the view frustum are culled by view frustum clipping to reduce unnecessary rendering tasks. An intersection test between the bounding box and the view frustum is used as a fast determination method, and the mesh list is clipped in parallel blocks by thread. For scene 3, this significantly reduces the number of triangles entering the raster stage, improving overall throughput.

1.3.2 Tile-Based Parallel Rendering (Single Mesh)

Tasks are assigned based on screen tiles, calculating a tile mesh (tilesX, tilesY). Each tile generates a job, which iterates through the triangles of the entire mesh, but only limits the write area within triangle::drawMT using ScissorRect.

All tile jobs are batched and sent to a thread pool for parallel execution.

Each tile job traverses all triangles of the mesh, limiting the write area only using scissor regions. The parallel granularity is tile-based, not triangle or mesh-based.

```

1  const int tilesX = (W + MT_TILE_W - 1) /
    MT_TILE_W;
2  const int tilesY = (H + MT_TILE_H - 1) /
    MT_TILE_H;
3
4  std::vector<ThreadPool::Job> batch;
5  batch.reserve(tilesX * tilesY);
6
7  for (int ty = 0; ty < tilesY; ++ty) {
8      for (int tx = 0; tx < tilesX; ++tx) {
9          ScissorRect sc{
10             tx * MT_TILE_W,
11             ty * MT_TILE_H,
12             std::min((tx + 1) * MT_TILE_W, W),
13             std::min((ty + 1) * MT_TILE_H, H)
14         };
15
16         batch.emplace_back([&, sc, vCache, lp,
17             mesh]() {
18             for (auto& ind : mesh->triangles) {

```

```

18         if (std::fabs(vCache->p.z[ind.v] 28
[0]) > 1.0f || std::fabs(vCache->p.z[ind.v] 29
[1]) > 1.0f || std::fabs(vCache->p.z[ind.v] 30
[2]) > 1.0f) { 31
19             continue; 32
20         } 33
21         triangle::drawMT(renderer, *
vCache, ind, *lp, sc);
22     }
23     });
24 }
25 }

```

Listing 5: Message Header Struct

1.3.3 Tile-Based Parallel Rendering (Multi-Mesh Scene)

Each mesh undergoes preprocessing and view frustum clipping. Then, VertexSOA and LightSIMD models are generated for each mesh to facilitate SIMD calculations. The results are stored in renderData (a list of meshes). Tasks are distributed by screen tile, using the same tile mesh and job generation method as Single Mesh, but each tile job iterates through the triangles of all meshes in renderData, again controlling the write range using the scissor in triangle::drawMT. All tile jobs are submitted to the thread pool in batches at once.

Each tile iterates through all triangles of all meshes in the entire scene, but the final pixel write is limited by the scissor.

```

1     auto renderData = std::make_shared<std::
vector<MeshRenderData>>>();
2     renderData->reserve(scene.size());
3
4     for (auto* mesh : scene) {
5
6     #if OPT_FRUSTUM_CULLING
7         vec4 boundsCenter;
8         float boundsRadius = 0.0f;
9         mesh->getBoundsSphere(boundsCenter,
boundsRadius);
10        matrix view = camera * mesh->world;
11        vec4 centerView = view * boundsCenter;
12        float radiusView = boundsRadius *
maxScaleFromMatrix(mesh->world);
13
14        if (!renderer.sphereInFrustumView(
centerView, radiusView)) {
15            continue;
16        }
17        matrix p = renderer.perspective * view;
18    #else
19        matrix p = renderer.perspective * camera
* mesh->world;
20    #endif
21
22        MeshRenderData data;
23        data.mesh = mesh;
24        data.vCache = std::make_shared<VertexSOA>
>();
25        data.lp = std::make_shared<avx2::
LightSIMD>(L, mesh->ka, mesh->kd);
26
27        mesh->preProcessVertexCache(p,
static_cast<float>(W), static_cast<float>(H)
, *data.vCache);

```

Listing 6: Message Header Struct

1.4 Conclusion and Future Work

I have currently applied multiple optimizations to the rasterizer, including vertex caching, early Z-testing, back-face culling, edge-function and inverse-area optimizations, AVX SIMD acceleration, as well as multi-threading and tile-based rendering. I defined compile-time switches for these optimizations in Macros.h.

In addition, the designed Scene 3 effectively validates the impact of frustum culling optimizations. Together with Scene 1 and Scene 2, these three scenes represent different levels of complexity, making the performance evaluation of the renderer more comprehensive.

Currently, multithreading is based on fixed tile rendering. In the future, I may explore dynamic task distribution, work-stealing, and other methods to improve load balancing in complex scenarios.

The current multithreading implementation is based on fixed tile-based rendering. In the future, it may be worth exploring dynamic task distribution and work-stealing strategies to further improve load balancing in complex scenes.

2 Chat Room

2.1 Overview

This project implemented a client/server(C/S) chat system from scratch, with a custom protocol, concurrent processing, and complete chat functionality. The system provides reliable message transmission and multi-threaded processing capabilities, supporting group chat, private chat, system notifications, and online users synchronization. The main achievements are summarized as follows:

1. Complete communication protocol and message model:

The system defines message types, message encoding, and a unified message header containing length and type fields, ensuring scalable message transmission formats.

2. Stable TCP message framing and parsing mechanism:

By using a TCPBuffer and the message parsing workflow in BaseSession, the system achieves reliable packet reception and a clear decoupling between network IO and business logic.

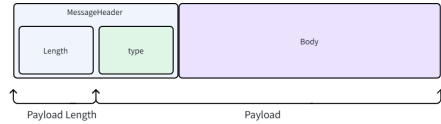


Figure 2: Message Header Structure

3. Multi-threaded and concurrent event-driven architecture:

The server is composed of an accept thread, an event thread, and session IO threads. The client consists of an IO thread and an event thread. With these designs, they provide the system's concurrent chat processing capabilities.

2.2 Networking

2.2.1 Message Passing

Message Protocol and Model

The system defines a unified communication protocol and message model for both the server and the client. Each interaction between the client and the server uses a custom protocol with a unified message header and a variable-length message body. The message header consists of two fields: length and type.

1. **length 4 bits, (uint32):** Representing the total payload size (type + body). This value is calculated during the encode() stage and written into the TCP stream.
2. **type 2 bits, (uint16):** Representing the business message type.
3. **body:** Variable-length content, based on the encoding result of different business messages.

This design ensures that message boundaries can be accurately identified within the TCP byte stream. The sender packages messages according to the protocol and transmits it via TCP. The receiver then parses the TCP byte stream following the same protocol and performs the corresponding operations.

```

1 struct Message {
2     MessageType type{};
3     string body;
4
5     // length is pushed during encoding.
6     vector<char> encode() const {
7         vector<char> packet;
8         packet.insert(length);
9         packet.insert(type);
10        packet.insert(body);
11        return packet;
12    }
13
14    BusinessMsg decode() const {
15        ...
16    }
17 }

```

Listing 7: Message Header Struct

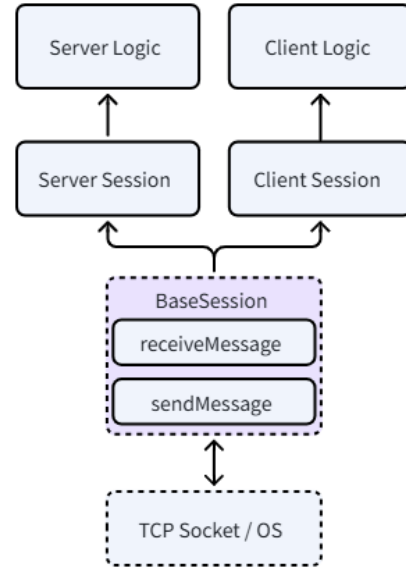


Figure 3: BaseSession Class Design

2.2.2 TCP Message Framing

BaseSession is an abstract middleware layer between the protocol and business logic, and is responsible for the following:

- Representing an established TCP connection
- Converting the TCP byte stream into structured messages (Message)
- Exposing event-driven message callbacks (onMessage) to the upper layer

In this system, the Session classes (ServerSession / ClientSession) provide general send and receive function.

- BaseSession::handleData() passes the raw byte stream to TcpBuffer for message framing
- BaseSession::sendMessage() encodes the message and sends it directly to the socket.

```

1 class BaseSession {
2 public:
3     BaseSession(SOCKET socket)
4         : socket(socket) {}
5
6     void handleData(const char* data, size_t len) {
7         readBuffer.push(data, len);
8         Message msg;
9         while (readBuffer.tryPopMessage(msg)) {
10             onMessage(msg);
11         }
12     }
13
14     void sendMessage(const Message& message) {
15         auto data = message.encode();
16         send(socket, data.data(), (int) data.size(), 0);
17     }
18 }

```

```

17     }
18
19 protected:
20     SOCKET socket;
21     TcpBuffer readBuffer;
22     virtual void onMessage(const Message&
23         message) = 0;

```

Listing 8: BaseSession Class

Sending When the client/server is preparing to send a message, all message types are uniformly encapsulated into a Message structure. When `Message::encode()` is called, the entire content of the message is encoded. Then, the TCP byte stream is sent directly using `send()`.

```

1 struct PrivateChat {
2     Chatter sender;
3     Chatter receiver;
4     string content;
5     string encode() const { ... }
6     static PrivateChat decode(const string& data
7         ) { ... }
8 }
9 // business side want to send private message
10 Client {
11     // encode what business side want to send
12     Chatter receiver(receiverId, "default");
13     Chatter sender = currentUserOrEmpty();
14     PrivateChat pc{ sender, receiver, text };
15     string pcBody = pc.encode();
16     // encode private Message
17     Message msg{ MessageType::ChatPrivate,
18         pcBody };
19     auto data = message.encode();
20     send(socket, data.data(), (int)data.size(),
21         0);
22 }

```

Listing 9: General Sending Process

Receiving and Message Framing TCP is a streaming protocol, and we cannot guarantee that a complete message is received in a single read. Therefore, this system implements a custom `TcpBuffer` for buffering and message framing. The main process is as follows:

1. Read the message header (length + type) until it is full.
2. Determine whether the full payload has been received based on the length.
3. Parse the message and remove the corresponding bytes from the buffer.
4. Repeat this process to get all the complete messages.

This mechanism ensures a clear decoupling between network I/O and business logic: the socket is only responsible for sending and receiving byte streams, and the business processing only accepts complete messages.

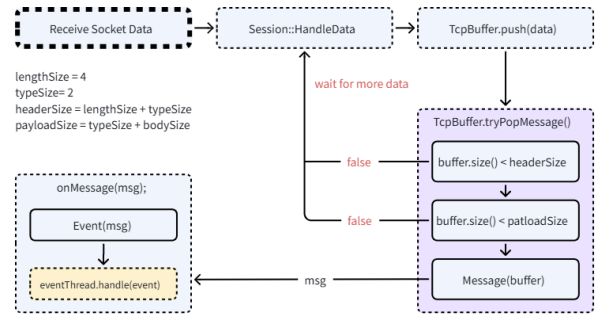


Figure 4: TcpBuffer Working Flow

```

1 TcpBuffer {
2     vector<char> buffer;
3
4     // add socket buffer into buffer
5     void push(const char* data, size_t size) {}
6
7     // Check buffer whether match a full message
8     // [ Length(4) | Type(2) | Body(N) ]
9     // Length = sizeof(Type) + sizeof(Body)
10    bool tryPopMessage(Message& outMsg) {
11        if (buffer.size < HEADER_SIZE) { return
12            false; }
13
14        // Parse length (4 bits)
15        totalPayloadSize = ntohl(buffer[0..4])
16
17        // cant get full (type + body) info
18        if (buffer.size < LENGTH_FIELD_SIZE +
19            totalPayloadSize) {
20            return false;
21        }
22
23        // Parse type (2 bits)
24        outMsg.type = (MessageType)ntohl(buffer
25            [4..6])
26
27        // Parse true body content
28
29        bodySize = totalPayloadSize -
30            TYPE_FIELD_SIZE
31
32        if (bodySize > 0) {
33            outMsg.body.assign(buffer[
34                HEADER_SIZE .. HEADER_SIZE + bodySize]);
35        } else {
36            outMsg.body.clear();
37        }
38
39        buffer.erase(consumedSize);
40        return true
41    }
42 }

```

Listing 10: TcpBuffer Class

2.2.3 Server Workflow

Threading Model The server adopts a two-layer threading structure:

- Accept Thread
 - Listening at the port to accept new connections.
 - Creates a ServerSession for each new connection and starts the IO thread for that session.

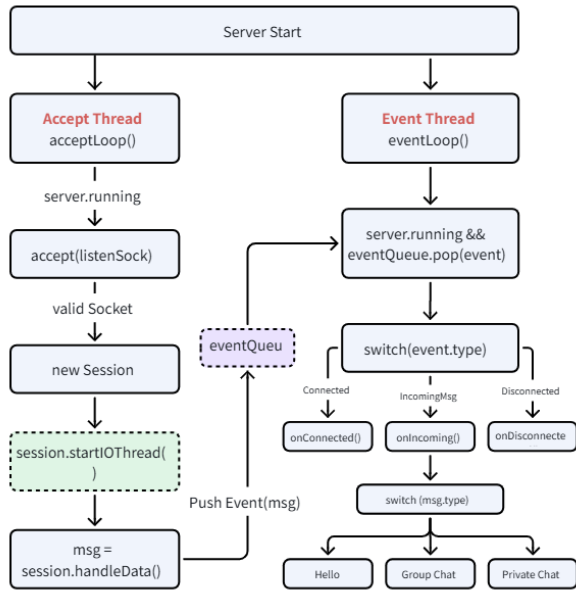


Figure 5: Server Threading Model

- Pushes a new connection event into the event queue.

• Event Thread

- Retrieves events from the ThreadSafeQueue (Connected / IncomingMsg / Disconnected).
- Invokes the corresponding logic to handle connection, incoming messages, or disconnection.

Session and Message Reception

Each client connection on the server corresponds to a ServerSession:

1. The session starts with an internal I/O thread that continuously calls `recv()`.
2. Received byte streams are passed to `BaseSession::handleData()` to be parsed into complete messages.
3. The parsed message is encapsulated into a `ServerEvent::IncomingMsg` and pushed to the event queue.
4. When the connection is closed, a `Disconnected` event is generated and dispatched.

Message Handling Logic

In this system, the server's core message-handling logic is implemented in `Server::onIncoming()`. Message types cover core functions such as login, group chat, private chat, online/offline broadcasts, and system notifications:

```
1 enum class MessageType : uint16_t {
2     Hello=1, // client->server:[name]
3     Welcome=2, // server->client:[chatter]
4     ChatGroup=3, // client<->server:[chatter][content]
```

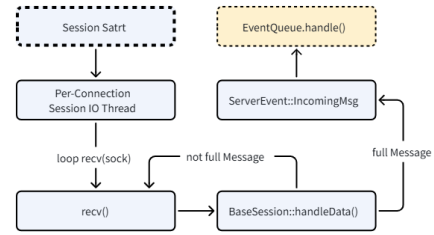


Figure 6: Session IO Thread Working Flow

```
5 ChatPrivate=4, // client->target:[chatter][chatter][content]
6 UserJoin=5, // server->all:[chatter][text]
7 UserLeave=6, // server->all:[chatter][text]
8 SystemMessage=7, // server->client:[chatter][text]
9 };
```

Listing 11: Message Types

The server processes different message types as follows:

1. MessageType Hello

- Generates a unique UID based on the client's username.
- Binds the user information to the session.
- Sends back a Welcome message (including the current online user list).
- Broadcasts a UserJoin message to notify all online users.

```
1 case MessageType::Hello: {
2     HelloMsg hello = HelloMsg::decode(msg.body);
3     // generate uid from server
4     Chatter me(hello.name);
5     session->setUser(me);
6     _uidToSid[me.chatterID] = sid;
7
8     // Send Welcome to client (with current users)
9     WelcomeMsg welcome;
10    welcome.me = me;
11    for (auto& [id, s] : _sessions) {
12        if (s->user() && s->user()->chatterID != me.chatterID) {
13            welcome.allUsers.push_back(*s->user());
14        }
15    }
16    Message welcomePacket{
17        MessageType::Welcome,
18        welcome.encode()
19    };
20    session->sendMessage(welcomePacket);
21
22    // broadcast new user
23    UserJoin join { me, "joined." };
24    Message joinMsg { MessageType::UserJoin, join.encode() };
25
26    broadcast(joinMsg);
27    break;
28 }
29
```

```

30 void broadcast(const Message& msg) {
31     auto data = msg.encode();
32     for (auto& [sid, s] : _sessions) {
33         ::send(s->getSocket(), data.data(),
34             (int)data.size(), 0);
35     }
36 }

```

Listing 12: Message Hello

2. MessageType ChatGroup

- Uses the session's associated user identity as the authoritative sender, correcting the sender information if necessary.
- Broadcasts the message to all online sessions.

```

1 case MessageType::ChatGroup: {
2     if (!session->user()) break;
3     GroupChat gc = GroupChat::decode(msg.
4         body);
5     gc.sender = *session->user();
6     Message out {
7         MessageType::ChatGroup,
8         gc.encode()
9     };
10    broadcast(out);
11    break;
12 }

```

Listing 13: Message ChatGroup

3. MessageType ChatPrivate

- Look up the target session by the recipient ID.
- If found: forwards the message to the target user and echoes it back to the sender.
- If not found: sends a SystemMessage indicating that the recipient is offline.

```

1 case MessageType::ChatPrivate: {
2     if (!session->user()) break;
3     PrivateChat pc = PrivateChat::decode(msg.
4         body);
5     pc.sender = *session->user();
6
7     auto itSid = _uidToSid
8         .find(pc.receiver.chatterID);
9     if (itSid == _uidToSid.end()) {
10        // offline
11        SystemMessage event{
12            pc.receiver,
13            "Receiver not online"
14        };
15        Message systemMessage {
16            MessageType::SystemMessage,
17            event.encode()
18        };
19        // send system message to sender
20        session->sendMessage(systemMessage);
21        break;
22    }
23
24    auto itTarget = _sessions.find(itSid->
25        second);
26    if (itTarget == _sessions.end()) break;
27
28    Message out{ MessageType::ChatPrivate,
29        pc.encode() };
30    itTarget->second->sendMessage(out);
31    session->sendMessage(out);
32 }

```

```

29     break;
30 };

```

Listing 14: Message ChatPrivate

4. MessageType Disconnection

- If the session corresponds to a logged-in user, broadcasts a UserLeave message.
- Cleans up the mapping between user IDs and sessions.

```

1 void onDisconnected(SessionId sid) {
2     auto it = _sessions.find(sid);
3     if (it == _sessions.end()) return;
4
5     // if sessionid belong to a login user,
6     broadcast left
7     if (it->second->user()) {
8         SystemMessage ev{
9             *it->second->user(),
10            " left."
11        };
12        Message msg{
13            MessageType::UserLeave,
14            ev.encode()
15        };
16        broadcast(msg);
17        // delete user
18        _uidToSid.erase(it->second->user()->
19            chatterID);
20    }
21
22    it->second->stop();
23    _sessions.erase(it);
24 }

```

Listing 15: Message Disconnected

2.2.4 Client Workflow

Connection and Login

1. Establish a TCP connection to the server.
2. Create a ClientSession and start the IO thread.
3. Immediately send a Hello(name) Message to register login information.

```

1 bool start(string& userName) {
2     ...
3     Message hello{
4         MessageType::Hello,
5         HelloMsg{ userName }.encode()
6     };
7     _session->sendMessage(hello);
8     ...
9 }

```

Listing 16: Client Connecting Message

Sending Messages

The client can actively send two types of business messages:

- Group chat: `sendGroupMessage()` assembles a ChatGroup message and sends it.

```

1 struct GroupChat {
2     Chatter sender;
3     string content;
4 }

```

Listing 17: GroupChat Structure

- Private chat: sendPrivateMessage() assembles a ChatPrivate message and sends it.

```

1 struct PrivateChat {
2     Chatter sender;
3     Chatter receiver;
4     string content;
5 }

```

Listing 18: PrivateChat Structure

Receiving and Processing Messages

The client-side receiving workflow is divided into two layers:

- ClientSession I/O thread
 - Continuously calls recv() and parses complete messages.
 - Wraps messages into ClientEvent::IncomingMsg and pushes them into a event queue.
- Client event thread thread
 - Retrieves messages from the queue and puts them into incomingQueue.
 - processIncoming() handles all messages uniformly and updates local state.

Local State Management

The client maintains:

- Its own identity information (uid and name).
- A mapping of online users (uid : name).
- Message history (for group chat and private chat windows).

```

1 struct ChatMsg {
2     enum class Type { ... };
3     bool fromMe;
4     string text;
5     Chatter sender;
6     Type type = Normal;
7 };
8
9 class ChatModel {
10 public:
11     enum class LoginType { ... };
12     bool isConnected = false;
13     string nameBuffer;
14     string inputBuffer;
15     LoginType state = Default;
16     Chatter me;
17
18     vector<PrivateChatWindow> privateChats;
19     vector<ChatMsg> mainChatMessages;
20 }
21
22 class Client {
23     Chatter _self;

```

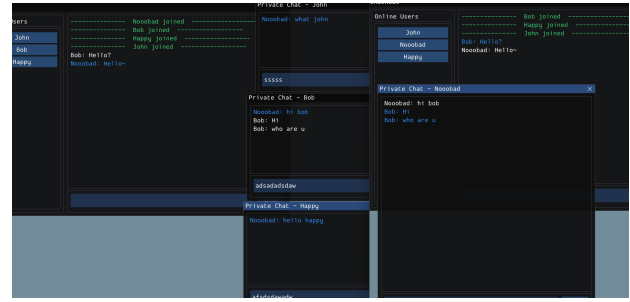


Figure 7: All Chats

```

24 unordered_map<string, string> _idToName;
25 }

```

Listing 19: Client Chat Model

Upon receiving different message types, the client updates its state accordingly:

- Welcome: initializes the client's identity and on-line user list.
- UserJoin / UserLeave: updates the online list and inserts system notifications.
- ChatGroup / ChatPrivate / SystemMessage: appends messages to the message history.

```

1 void handleIncoming(message, model) {
2     switch (msg.type) {
3     case Welcome(welMsg):
4         model.me = welMsg.me;
5         for (user : welMsg.allUsers) {
6             _idToName[user.id] = user.name;
7         }
8     case ChatGroup(gcMsg):
9         model.mainChat.push_back(gcMsg);
10    case ChatPrivate(pcMsg):
11        model.pcs.push_back(pcMsg);
12    case UserJoin(msg): ...
13    case UserLeave(msg): ...
14    }
15 }

```

Listing 20: Client Message Handler

2.3 GUI

2.3.1 Overview

The system implements a simple chat room interface using ImGui and FMod, consisting of the following components:

- Login Window
- Main Chat Room Window
- Private Chat Window
- Audio Player

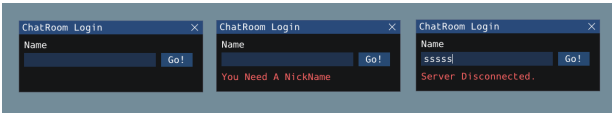


Figure 8: Login Window With Status

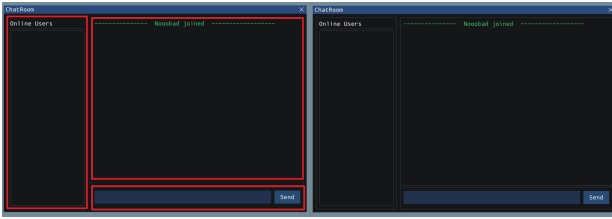


Figure 9: Main Chatroom Modular Layout

2.3.2 Login Window

My login UI is driven by a ChatModel state machine, while the Client serves as the underlying networking module, providing synchronous startup and runtime status queries. It provides a nickname input field and a "Go!" button for establishing a connection and entering the main chat room and includes a status message area to display results such as successful connection, connection failure, or missing nickname.

Currently, state convergence and basic error feedback are achieved through a synchronous connection process combined with per-frame polling of `isRunning()`. However, due to the synchronous nature of the network requests, error state updates are rigid and cannot be delivered asynchronously. While this approach is simple to implement and has deterministic behavior, it causes the UI interaction to be blocked by network state and limits the granularity of state updates.

```

1 if (ImGui::Button("Go!", ImVec2(60, 0))) {
2     if (model.nameBuffer.empty()) {
3         model.state = EmptyName;
4     }
5     else {
6         // Start Connect to Server From UI
7         model.isConnected = client.start(model.nameBuffer);
8         model.openMainChat = true;
9         model.state = model.isConnected ? ChatModel::LoginType::Connected : ChatModel::LoginType::NetError;
10    }
11 }

```

Listing 21: Login UI

2.3.3 Main Chat Room Window

Modular Layout

The main chat room adopts a component-based design, splitting the UI into three functionally independent sub areas to ensure high code maintainability and high interface availability:

- Sidebar(OnlineList): Located on the left side of the interface, displaying a real-time list of current



Figure 10: Online List With Another User

online users (excluding local user). Scrolling is achieved through independent child windows, ensuring that adding to the list does not affect the main view.

```

1 ImGui::BeginChild("UserListScroll", ...);
2     if (ImGui::BeginTable("UserListTable", ...)) {
3         const auto users = client.getCurrentOnlineUser();
4         for (const auto& user : users) {
5             if (user.chatterID == model.me.chatterID) continue;
6             string label = user.chatterName;
7             ImGui::TableNextColumn();
8             ImGui::PushStyleVar(...);
9             if (ImGui::Button(label.c_str(), ...)) {
10                 model.openPrivateChatFor(user);
11             }
12             ImGui::PopStyleVar();
13         }
14         ImGui::EndTable();
15     }
16 ImGui::EndChild();

```

Listing 22: OnlineList UI

- Main View(MessageList): Occupies the core right-hand area. It uses responsive height calculation, automatically filling all remaining space above the input boxes to display a waterfall-style information flow that mixes system notifications and user chat.

```

1 ImGui::BeginChild("MessageList", ...);
2 for (const auto& msg : messages) {
3     if (msg.type != Normal) {
4         drawSystemMessage();
5     }

```

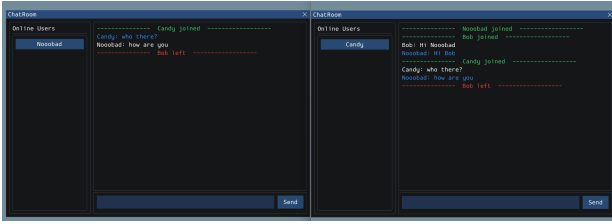


Figure 11: Message List with Different Message

```

6     string info = msg.sender.chatterName
7     if (msg.fromMe) {
8         ImGui::PushStyleColor (...);
9         ImGui::TextUnformatted (info.c_str());
10        ;
11        ImGui::PopStyleColor ();
12    }
13    else {
14        ImGui::TextUnformatted (info.c_str());
15    }
16 }
17 ImGui::EndChild ();

```

Listing 23: MessageList UI

- Bottom Area(InputArea): Fixed at the bottom of the window, integrating text input and sending functions.

```

1 ImGui::BeginChild("InputArea", ...);
2 bool send = false;
3 if (ImGui::InputText("##chat_input",...)) {
4     send = true;
5 }
6 ImGui::SameLine();
7 if (ImGui::Button("Send", ...))
8     send = true;
9 ImGui::EndChild();

```

Listing 24: InputArea UI

Interaction Logic and Experience Optimization

- Visual Semantic: The UI uses color to layer information. System messages (such as user join or leave) are highlighted in green and red with a line; user-defined messages are highlighted in blue. This differentiation allows users to instantly locate key events in the information stream.

```

1 if (msg.type != ChatMsg::Type::Normal) {
2     string banner;
3     ImVec4 color = ImVec4(...);
4     if (msg.type == SystemJoin) {
5         banner = format("-- {} joined --",
6             msg.text);
7         color = ImVec4(0.25f, 0.80f, 0.45f,
8             1.0f);
9     }
10    else if (msg.type == SystemLeave) {
11        ...
12    }
13    else {
14        ...
15    }
16    ImGui::PushStyleColor(ImGuiCol_Text,
17        color);

```

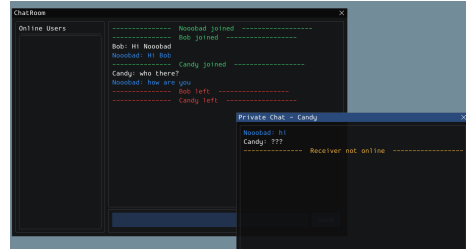


Figure 12: Colourful Messages

```

15 ImGui::TextUnformatted (banner.c_str());
16 ImGui::PopStyleColor ();
17 continue;
18 }

```

Listing 25: Visual Semantic UI

- EnterReturnsTrue: To align with the operating habits of mainstream IM software, the input box uses the `ImGuiInputTextFlags_EnterReturnsTrue` flag. This allows users to directly trigger the sending logic with the "Enter" key, without frequently switching between the keyboard and mouse.
- SetKeyboardFocusHere: After a message is sent, the program will force the input focus to be re-locked in the input box using `ImGui::SetKeyboardFocusHere(-1)`. This design eliminates the pause in operation after sending.

```

1 if (ImGui::InputText("##chat_input", &model.
    inputBuffer,
    ImGuiInputTextFlags_EnterReturnsTrue))
2     {
3         send = true;
4     }
5     ...
6     if (send) {
7         if (!model.inputBuffer.empty()) {
8             client.sendGroupMessage(model.
9                 inputBuffer);
10            model.inputBuffer.clear();
11        }
12        ImGui::SetKeyboardFocusHere(-1);
13    }

```

Listing 26: Better Input

Communication Mechanism between UI and Client

The GUI layer and the underlying Client network module interact through a combination of state-polling and event-driven mechanisms:

- State Synchronization: The interface calls `client.getCurrentOnlineUser()` every frame to retrieve the latest user list in a non-blocking manner. `getCurrentOnlineUser()` requests all online users upon the first connection and then manually synchronizes the online user list locally based on user join/leave notifications.

```

1 vector<Chatter> getCurrentOnlineUser() {
2     vector<Chatter> result;

```

```

3     result.reserve(_idToName.size());
4     for (const auto& [id, name] : _idToName)
5     {
6         result.emplace_back(id, name);
7     }
8     return result;
9 }

```

Listing 27: State Synchronization

- **Public Broadcast:** When a send action is triggered, the UI submits the string in `model.inputBuffer` to `client.sendGroupMessage()`.
- **Lifecycle Closure:** A crucial destruction logic is added to the interface-once the main chat window is closed (`wasOpen` becomes `false`), the system automatically calls `client.stop()` and clears all Model states. This interface-driven lifecycle design ensures that the network connection is gracefully disconnected when the window is closed, avoiding background zombie processes.

```

1     if (wasOpen && !model.openMainChat) {
2         client.stop();
3         model.isConnected = false;
4         model.state = ChatModel::LoginType::
5         Disconnected;
6         model.openPrivateChat = false;
7         model.openLogin = false;
8         model.allowLoginWindow = false;
9         for (auto& chat : model.privateChats
10     ) {
11         chat.open = false;
12     }
13     model.removeClosedPrivateChats();
14 }

```

Listing 28: Lifecycle Closure

2.3.4 Private Chat Window

Multi-Instance Concurrency and Context Isolation

- **Multi-Instance Support:** Unlike the globally unique group chat window, the private chat window is rendered using dynamic iteration. By traversing the `model.privateChats` container, the system can pop up multiple private chat windows simultaneously. To prevent ImGui component ID conflicts, the design cleverly uses `ImGui::PushID(chat.user.chatterID.c_str())`, ensuring that each window's input fields and buttons have independent ID fields.

```

1     for (auto& chat : model.privateChats) {
2         ImGui::PushID(chat.user.chatterID.c_str());
3         ...
4         ImGui::PopID();
5     }

```

Listing 29: ImGui ID conflicts Solution

- **Visual Consistency:** The private chat window reuses the `drawMessageList` component, maintaining the same visual language as the main group

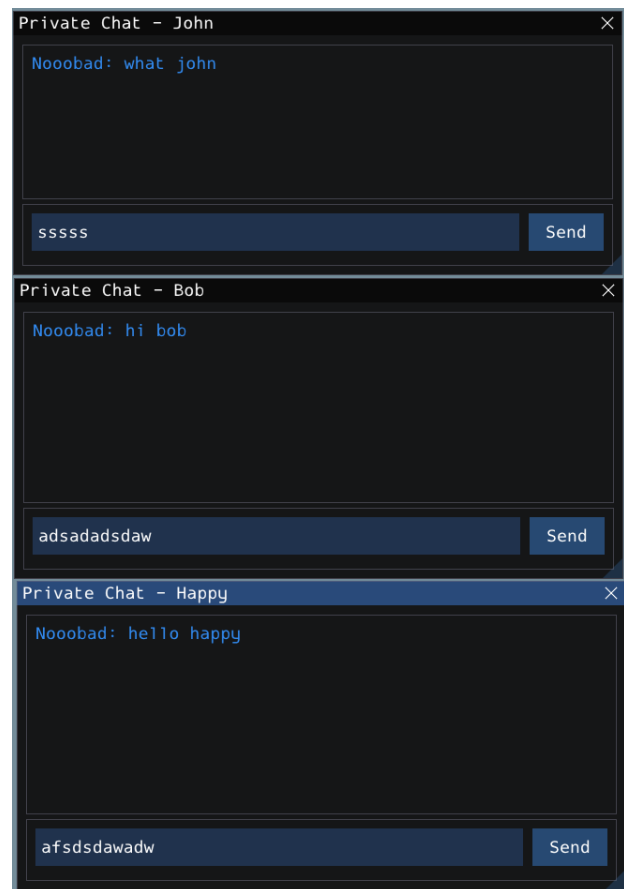


Figure 13: Multi Private Chat Window

chat (such as highlighting its own messages in blue, uniform rounded corners, etc.). This high degree of code reuse not only reduces maintenance costs but also lowers the user's learning curve.

Communication Mechanism between UI and Client

- **State-driven window lifecycle:** Closing the window directly modifies the `chat.open` flag. An updated flag is introduced in the code; once a user closes a private chat window, `model.removeClosedPrivateChats()` is immediately called. This design enables real-time memory management at the UI layer, preventing the storage of too many inactive session instances in the background.
- **Efficient Interaction:** The private chat window also integrates the `ImGuiInputTextFlags_EnterReturnsTrue` and `SetKeyboardFocusHere` mechanisms. This ensures that users can still enjoy a enjoyable input experience in private chat.
- **Private Chat Initiation:** When a user clicks on a name in the sidebar, the UI does not directly send a network packet but modifies the `ChatModel` state, driven by the state machine, and opens the corresponding private chat window.

```

1 void openPrivateChatFor(const Chatter& user)
2 {
3     for (auto& chat : privateChats) {
4         if (chat.user.chatterID == user.chatterID) {
5             chat.open = true;
6             chat.inputBuffer.clear();
7             chat.messages.clear();
8             return;
9         }
10    }
11    PrivateChatWindow window{};
12    window.user = user;
13    privateChats.push_back(move(window));
14 }
```

Listing 30: *Private Chat Initiation*

- **Decoupled Input Area:** `drawPrivateInputArea` specifically accepts `PrivateChatWindow&` references. This instance-specific parameter passing ensures that the input buffer data is absolutely independent when multiple windows are running concurrently.

2.3.5 Audio Player

A simple audio player is implemented and configured with two different notification sound effects. One is played when another user comes online, and the other is played when a message is received from someone other than the local user.

2.4 Conclusion and Future Work

2.4.1 Conclusion

Based on the existing implementation, this project has completed a working WinSock multi-client chat room with a user interface: the server can accept multiple connections and handle broadcast/private chat routing; the client can connect to the server and display the online list, main chat room, and private chat window through ImGui. The code structure design also feels just right. It avoids the over-design issues that occurred previously. It effectively decouples network listening, data processing, and UI updates. A bad design is the lack of asynchronous network request updates. Later, when red text flashed during each login, I realized that the login status indicator text was always red before establishing a connection. This is easily solved, but the best solution is definitely to introduce asynchronous network requests, something I hadn't considered at all during development.

2.4.2 Future Work

- Introduce a reconnection mechanism after disconnection.
- Add lightweight multimedia message support, such as emoji/image messages, as meme is important.
- Introduces configuration files (server address, port, UI parameters, etc.) to reduce hard coding.
- Online user list supports searching and status marking (idle/busy/offline placeholder).
- Unread private chat message badge in user list.
- Supports more room talk: multiple group chat rooms and invitation/join/leave logic.